

Task 1 — Launch Two Web Servers

Launch two EC2 instances.

Example

- WebServer-1
- WebServer-2

Use an appropriate current Amazon Linux image available in your AWS account.

Install Apache

Connect to each instance and run:

- `sudo dnf update -y`
- `sudo dnf install httpd -y`

Start Apache:

- `sudo systemctl enable --now httpd`

Create different test pages.

Server 1

- `echo "Hello from Web Server 1" | sudo tee /var/www/html/index.html`

Server 2

- `echo "Hello from Web Server 2" | sudo tee /var/www/html/index.html`

Configure the security group to permit HTTP traffic as required for the lab.

Task 2 — Create an Application Load Balancer

Steps

1. Open EC2 → Load Balancers.
2. Select Create Load Balancer.
3. Choose Application Load Balancer.
4. Name it:
 - InstaVibe-ALB
5. Select Internet-facing.

6. Choose the appropriate VPC.
7. Select suitable subnets in different Availability Zones.
8. Configure a listener:
 - HTTP : 80
9. Create a target group.
10. Target type:
 - Instances
11. Register:
 - WebServer-1
 - WebServer-2
12. Configure the target group health check.
13. Create the load balancer.

Task 3 — Custom /health Health Check

Create the endpoint

On each server:

- `echo "OK" | sudo tee /var/www/html/health`

Test:

- `http://SERVER-IP/health`

Expected:

- OK

Configure ALB

Go to:

- EC2 → Target Groups → Health checks

Set:

- Protocol: HTTP
- Path: /health

- Port: Traffic port

The expected successful HTTP response should be a healthy response such as 200 OK.

Task 4 — Simulate Server Failure

Steps

1. Verify both instances are healthy.
2. Open the ALB DNS name.
3. Stop one EC2 instance.
4. Wait for the health check to detect the failure.
5. Refresh the ALB URL.

Task 5 — ALB vs NLB vs Classic Load Balancer

Feature	ALB	NLB	Classic LB
Main layer	Layer 7	Layer 4	Layer 4/7
Protocols	HTTP/HTTPS	TCP/UDP/TLS	HTTP/HTTPS/TCP
Best for	Web applications/APIs	High-performance network workloads	Older applications
Routing	Host/path-based routing	Connection/flow based	Basic load balancing
Performance	Excellent for HTTP applications	Very high performance/low latency	Legacy
Modern recommendation	Common choice for web apps	Common choice for network-level traffic	Generally replaced by

			newer options
--	--	--	------------------